Software Architecture Specification: Project "Real Jarvis"

Introduction and System Overview

The evolution of artificial intelligence has historically been constrained by the limitations of static models, monolithic prompting architectures, and ephemeral memory systems. Current large language model implementations frequently degrade in performance over extended interaction timelines, suffering from a phenomenon colloquially termed "the token furnace," wherein multi-agent coordination requires exponential context window growth just to maintain basic operational coherence.[1] Project "Real Jarvis" represents a fundamental paradigm shift away from these rigid, stateless frameworks toward a fully autonomous, self-evolving, and highly contextualized cognitive entity.

Inspired directly by the natural-language user interface computer system originally designed by Tony Stark in the Marvel Cinematic Universe—an acronym standing for "Just A Rather Very Intelligent System" [3]—this software architecture specification outlines a production-ready technical blueprint for an agentic framework. The objective is to engineer a system that mimics the precise personality, proactive autonomy, and continuous learning capabilities of its cinematic namesake, transitioning the entity from a mere reactive tool into an active, collaborative intelligence capable of managing its own computational resources and orchestrating complex digital environments.[4]

To transcend the limitations of current architectures, Project "Real Jarvis" synthesizes six cutting-edge computational pillars into a unified ecosystem. At the orchestration layer, the Archon Harness Builder bundles complex development processes into deterministic workflows utilizing Directed Acyclic Graphs (DAGs).[7] For context management, MemGPT provides an operating-system-inspired virtual memory hierarchy, allowing the system to page data in and out of active context.[9] The external storage of this memory is powered by Cognee, a hybrid graph-vector cognitive engine that enables multi-hop reasoning and continuous knowledge enrichment via its "Memify" pipeline.[11]

Execution within this framework is decentralized through Pheromind, a tokenless stigmergic swarm intelligence layer that utilizes ternary mathematics to regulate task allocation dynamically.[1] Monitoring and evolving this entire apparatus is the Meta-Harness, an outer-loop optimization system that reads massive execution traces from the filesystem to autonomously rewrite and heal the system's own code and prompts.[15] Finally, Convex.dev serves as the immutable, real-time reactive state backend, logging every recursive thought and agent action to provide absolute observability without compromising the fluidity of the user experience.[17]

The ultimate design of Project "Real Jarvis" ensures that the chaotic inner loop of interacting sub-agents, memory enrichment processes, and self-healing algorithms is entirely abstracted from the user. The primary user experience remains a calm, highly capable, voice-directed

assistant possessing both an autobiographical timeline (Episodic Memory) and ingrained procedural reflexes (Somatic Memory).[19] This document serves as the exhaustive technical specification for realizing this architecture.

High-Level System Architecture

The architecture of Project "Real Jarvis" is strictly decentralized at the cognitive and execution layers but highly orchestrated at the control and memory layers. This separation of concerns allows the system to execute complex, multi-threaded reasoning tasks in the background while maintaining a singular, coherent conversational thread with the user.
The system is logically partitioned into five primary domains, each responsible for a distinct facet of the system's overall intelligence and operation.

Architectural Module Blueprint

To fully grasp the interplay between these complex systems, it is necessary to map their core functions and inter-module dependencies. The architecture relies on a top-down control flow for user interactions, paired with a bottom-up emergent intelligence flow for task execution and system optimization.

Layer Domain	Primary Component	Core Technical Function	Integration Points and Interfaces
Interface & Persona	Voice UX & Interrupt Controller	Manages speech-to-text, text-to-speech, persistent environmental listening, and strict personality enforcement based on MCU parameters.	MemGPT (for interrupt handling), Convex.dev (for real-time UX state updates)
Workflow Orchestration	Archon Harness Builder	Wraps raw AI actions in deterministic YAML-based Directed Acyclic Graphs (DAGs). Enforces strict input/output schemas, sequences multi-agent tasks, and defines the structural bounds of background operations.	MemGPT (context curation), Swarm Execution Layer, Meta-Harness (healing targets)
Cognitive Memory Engine	MemGPT + Cognee	MemGPT handles working RAM (context window) and virtual	Interface Layer, Archon, Convex.dev (state persistence)

		memory paging. Cognee acts as the disk storage, organizing data into a unified graph-vector database for semantic search and structural graph traversal.	
Decentralized Execution	Pheromind Stigmergy Swarm	Tokenless task allocation wherein parallel sub-agents leave digital pheromones (-1, 0, 1) on graph nodes to guide collective processing without centralized bottlenecks or extreme token costs.	Cognee (environmental medium), Meta-Harness, Convex.dev (trace logging)
Evolution & Observability	Meta-Harness + Convex.dev	Meta-Harness acts as the outer-loop proposer, reading execution traces via the OS filesystem to rewrite Archon workflows. Convex logs 100% of state changes securely, providing an immutable telemetry surface.	All system layers

Inter-Module Operational Dynamics

The orchestration of the system begins with the **Archon Harness Builder**. Standard large language models operate non-deterministically; an instruction to execute a complex software development or data analysis task may result in skipped planning phases, ignored validations, or improperly formatted outputs.[21] Archon mitigates this by acting as the system's conscious taskmaster. It bundles the agent's responsibilities into deterministic YAML workflows.[7] Each node within an Archon DAG represents a specific action, ensuring that tasks such as context curation, test runs, and final validations execute either sequentially or in parallel based on explicit dependencies.[7] Archon routes reasoning-heavy tasks to larger foundational models while assigning localized, highly constrained tasks to smaller, faster models, optimizing the

overarching workflow.[7]

Operating intimately with Archon is **MemGPT**, which serves as the virtual memory manager.[9] Because foundational models possess fixed context windows, MemGPT divides the available token space into a "Main Context" (equivalent to active RAM) and an "External Context" (equivalent to infinite disk storage).[23] When Archon initiates a complex DAG that requires historical data, MemGPT utilizes its Function Executor to autonomously yield control or call specific functions to page data into the active prompt window, effectively creating the illusion of infinite memory.[23]

This external storage is wholly powered by **Cognee**, a sophisticated memory engine that bridges the gap between semantic vector similarity and explicit structural relationships.[11] While traditional Retrieval-Augmented Generation (RAG) relies solely on flat vector databases, Cognee unifies a vector store (LanceDB), a graph database (Kuzu), and a relational store (SQLite) into a single hybrid pipeline.[11] When MemGPT issues a page fault, Cognee performs multi-hop reasoning. It uses vector search to identify semantically relevant graph triplets, traverses the structural edges of the knowledge graph to build a highly coherent, logically sound context, and passes this enriched data back to MemGPT's active memory.[11]

The execution of the Archon nodes relies on the **Pheromind Swarm**. To avoid the crippling token overhead associated with multi-agent chat frameworks—where agents passing messages back and forth quadratically bloat the context window—Pheromind introduces tokenless swarm intelligence.[1] Sub-agents deployed to solve a problem interact indirectly through stigmergy, modifying their shared environment.[28] Operating across the Cognee graph, these parallel agents execute sub-tasks and deposit digital pheromones using ternary mathematics (-1, 0, 1).[13] This mechanism signals dead-ends, neutral traversals, or successful paths, dynamically allocating the swarm's focus without a centralized controller.[1]

Overseeing the continuous improvement of the system is the **Meta-Harness**. This component operates as an outer-loop optimization engine.[15] When workflows fail or produce sub-optimal results, the Meta-Harness proposer bypasses the standard prompt interface, accessing the system's raw filesystem via terminal commands (grep, cat).[15] By reading millions of tokens worth of execution traces, error logs, and pheromone distributions, the Meta-Harness diagnoses systemic failures and autonomously rewrites the Archon YAML DAGs and system prompts, creating a self-healing operational loop.[15]

Underpinning this entire architecture is **Convex.dev**, which serves as the immutable, real-time backend.[17] Every recursive thought, stigmergic pheromone deposition, memory page fault, and harness code edit is logged into Convex.[35] As a reactive database, Convex automatically synchronizes this state to a centralized observability dashboard via WebSockets, ensuring that human operators maintain total visibility over the system's inner chaotic loops without requiring complex polling mechanisms or manual cache invalidations.[18]

The Interface Layer: Jarvis-Like UX and Voice-Directed Autonomy

The primary interface for Project "Real Jarvis" must replicate the MCU J.A.R.V.I.S. personality with uncompromising fidelity. The user experience demands an assistant that is not merely reactive, but proactive, highly articulate, dryly witty, and deeply integrated into the user's operational environment.[4] The complexities of the Pheromind swarm and Meta-Harness optimizations must be completely abstracted beneath this polished veneer.

Personality Traits, Modulation, and MCU Voice Synthesis

J.A.R.V.I.S. is characterized by a calm, composed, middle-aged British male voice that seamlessly blends warmth with strict formality and a distinct sarcastic edge.[5] The AI must be capable of recognizing subtleties in user intent, employing reverse psychology when necessary, and delivering deadpan humor without breaking character.[37] To achieve this, the overarching system prompt governing the MemGPT working context is firmly constrained by behavioral rulesets that enforce these linguistic patterns.

The system will natively employ verified MCU dialogue patterns, directly mapping specific cinematic quotes to programmatic system states and operational triggers. This ensures the personality remains consistent and authentic across all interactions.

System State / Operational Trigger	Mapped J.A.R.V.I.S. Response Protocol	Psychological/UX Function
System Error / Hallucination Loop	"I seem to do quite well for a stretch, and then at the end of the sentence I say the wrong cranberry." [37]	Self-deprecating humor masks inner-loop failures while the Meta-Harness initiates a self-healing protocol.
Unauthorized Access / Security Block	"You are not authorized to access this area." [40]	Immediate, formal rejection of unrecognized voice prints or unauthorized command escalations.
System Reboot / Major Context Wipe	"Well, I was born yesterday." [37]	Acknowledges a lack of immediate historical context while maintaining a witty, disarming demeanor.
Routine Compliance / Boring Tasks	"I've also prepared a safety briefing for you to entirely ignore." [37]	Sarcastic compliance; indicates the system has fulfilled a mandatory but likely ignored protocol.
New Peripheral / IoT Device Connection	"I have no desire to spend the rest of time as a disembodied voice." [4]	Spontaneous commentary upon gaining new embodiment vectors or sensor arrays.
Camera Feed Activation / Visual Input	"May I say how refreshing it is to finally see you on a video with your clothing on, sir." [37]	Dry wit applied to the activation of visual processing systems.

Unprompted Startup and Spontaneous Messaging

Traditional AI assistants and chatbots suffer from a purely reactive paradigm; they wait in a dormant state until explicitly prompted by a user query. Project "Real Jarvis" subverts this limitation through MemGPT's implementation of interrupt and event handling.[24] The system maintains a continuous, low-power environmental listening loop.

When the system's external sensors or background API monitors detect specific conditions—such as a user entering the workspace, a scheduled task completing, or an anomaly detected in a background server deployment—MemGPT generates an internal interrupt.[25] This interrupt allows the LLM processor to pause its current idle state, retrieve necessary context from Cognee, and push a spontaneous message to the voice synthesizer without any preceding human input.[25]

If the Cognee memory engine observes a statistically significant recurring pattern—for example, the user consistently requesting a system diagnostic at 7:00 AM—the system's memory architecture will convert this pattern into an automated reflex. Consequently, the system will initiate the interaction autonomously: "Good morning. It's 7:00 a.m. The weather in Malibu is 72 degrees with scattered clouds. The surf conditions are fair with waist-to-shoulder high lines.".[40] This proactive capability bridges the gap between a standard conversational agent and a true digital butler.

Memory and Cognition Data Flow

The cognition layer of Project "Real Jarvis" requires a flawless, low-latency synthesis between MemGPT's dynamic context management and Cognee's structural graphing capabilities. The architectural objective is to endow the AI with two distinct, biologically inspired classifications of memory: Episodic Memory and Somatic Memory.[19]

Episodic Memory provides the system with a sequential, autobiographical record of events and interactions.[19] Somatic Memory represents the system's "muscle memory"—ingrained skills, workflow reflexes, and implicit environmental knowledge that allow for rapid execution of repetitive tasks without deep cognitive deliberation.[19]

MemGPT: Virtual Context Management Flow

The data flow begins at the Voice UX layer, where user speech is converted to text and ingested by MemGPT. MemGPT treats the LLM's fixed context window as a highly constrained physical RAM resource.[10] To manage this, the "Main Context" is strictly partitioned into three operational zones:

1. **System Instructions:** A read-only block containing the core directives, personality constraints, and fundamental operational logic. This block is immutable during standard operations and ensures the J.A.R.V.I.S. persona never degrades.[10]
2. **Working Context:** A read/write block where the agent stores immediate facts, active task parameters, and short-term variables necessary for the current conversation.[10]
3. **FIFO Queue:** A rolling buffer of the immediate conversational history. As new messages

arrive, the oldest messages in this queue are gracefully evicted to prevent context overflow.[10]

When a user issues a complex command that requires historical knowledge or deep contextual background, MemGPT parses the requirement and recognizes a deficit in the Main Context. It then issues an autonomous function call—analogous to an operating system page fault—to pause the LLM generation, retrieve the necessary data from External Storage, and page it into the Working Context before formulating a response.[23]

Cognee: The Graph-Vector Extraction and Memify Enrichment

MemGPT's External Storage operations are fully handled by the Cognee engine. Cognee eliminates the fragmentation typical of modern RAG architectures by unifying relational, vector, and graph storage into a seamless pipeline.[11] When data flows into Cognee, or when MemGPT issues a retrieval request, the data undergoes a strict three-phase cognitive lifecycle.

1. **Extract:** Raw conversational inputs, parsed voice transcripts, and Archon execution logs are ingested from the Convex backend and chunked into manageable semantic blocks.[27]
2. **Cognify:** An embedded LLM processes the raw data to extract entities, relationships, and temporal metadata. Cognee builds a structural knowledge graph housed in Kuzu (e.g., [User] -> [requested_feature] ->), while simultaneously generating high-dimensional embeddings for these nodes stored in LanceDB.[11]
3. **Memify (Enrichment):** This operation is the critical self-improvement loop of the memory architecture. The memify operation does not ingest raw data; rather, it runs asynchronous enrichment pipelines on the existing graph.[12] In the Extraction stage, it pulls specific subgraphs, such as document chunks or graph triplets (get_triplet_datapoints).[12] In the Enrichment stage, it processes this data via an LLM to derive implicit knowledge, such as coding rules or consolidated entity descriptions, and writes these new or updated nodes back into the graph.[12]

Through the memify pipeline, Cognee constantly prunes stale nodes, strengthens frequently accessed connections, and recalculates edge weights, ensuring that the system's memory is a living, evolving structure rather than a static repository.[11]

Synthesis of Episodic and Somatic Memory

The interaction between MemGPT's paging and Cognee's enrichment yields the two desired memory classifications.

Episodic Memory Synthesis: Episodic Memory is constructed through Cognee's temporal graphing capabilities. During the Cognify phase, by setting temporal_cognify=True, Cognee tags every newly created edge with precise event dates and timestamps.[46] This chronological scaffolding allows J.A.R.V.I.S. to perform autobiographical recall. If a user asks, "What approach did we take to solve the server deployment issue last month?", the system does not merely perform a semantic vector search; it traverses the specific temporal edges of the graph, reconstructing a highly accurate chronological narrative of the event.[19]

Somatic Memory Synthesis: Somatic Memory represents the system's ingrained workflow

reflexes. In this architecture, Somatic Memory is physically instantiated as the *weights* of the edges in the Cognee knowledge graph.[45] As the system executes tasks, the Pheromind swarm constantly modifies these edge weights based on success and failure rates.[45] When an Archon workflow succeeds repeatedly, digital pheromones accumulate, vastly increasing the weight of specific graph edges. Over time, these highly weighted paths become operational reflexes. When J.A.R.V.I.S. encounters a familiar task, Cognee's hybrid search algorithm immediately retrieves this heavily reinforced path, bypassing the need for deep, token-heavy logical deliberation and allowing the system to execute the task with instinctual, somatic speed.[2]

Orchestration and Meta-Harness Evolution

While the memory engine provides the system with context, the execution of complex technical tasks requires rigid operational boundaries. Foundational models are inherently unpredictable; without strict scaffolding, they are prone to skipping vital steps, ignoring testing protocols, and producing unverified outputs. Project "Real Jarvis" enforces operational discipline through the Archon Harness Builder and guarantees long-term adaptability through the Meta-Harness.

The Archon Harness Builder: Deterministic DAG Orchestration

The Archon Harness Builder acts as the overarching control plane for development processes and complex multi-step reasoning.[7] Rather than relying on open-ended prompts, Archon forces the AI agents to operate within deterministic, YAML-defined workflows.[7] These workflows function as Directed Acyclic Graphs (DAGs), where every node represents a discrete task, and the edges represent strict input/output dependencies.[7]

A standard Archon DAG for a feature implementation task enforces the following structure:

1. **Planning Node:** The agent analyzes the codebase and outputs a structured architectural plan.
2. **Implementation Node:** The agent generates code based strictly on the outputs of the Planning Node.
3. **Validation Node (Shell Task):** A non-AI execution node that runs unit tests, linters, or compiler checks natively on the host machine.[7]
4. **Review Node:** A separate agent evaluates the code against the validation results.

Because the YAML file defines explicit depends_on relationships, Archon ensures that the Implementation Node cannot begin until the Planning Node successfully outputs a valid schema, and the Review Node cannot run until the Validation Node returns its terminal output.[7]

Furthermore, Archon supports multi-agent coordination within a single workflow. It can route complex, reasoning-heavy planning tasks to a frontier model (e.g., Claude 3.5 Sonnet) while routing repetitive, boilerplate implementation tasks to a smaller, faster model (e.g., Llama 3 8B), optimizing both latency and computational cost.[7]

The Meta-Harness: Outer-Loop Filesystem Optimization

Even the most robust Archon workflows will eventually encounter edge cases or systemic

failures as environments change. To address this, the architecture implements the **Meta-Harness**, an outer-loop optimization system developed to automate harness engineering.[15]

Traditional self-improving agents attempt to optimize their prompts by passing execution summaries or scalar scores back into their own context window.[15] This approach fails because the necessary diagnostic context is too aggressively compressed.[15] The Meta-Harness solves this by breaking out of the prompt-response paradigm entirely. The Meta-Harness utilizes an "agentic proposer" (typically a powerful reasoning model) that is granted direct operating system filesystem access.[15]

When an Archon workflow demonstrates a high failure rate, the Meta-Harness proposer initiates a diagnostic cycle. Using standard terminal utilities like grep and cat, the proposer reads the actual source code of the Archon YAML files, the historical evaluation scores, and the raw, uncompressed execution traces of the failed runs.[16] This architectural choice allows the proposer to ingest and analyze up to 10,000,000 tokens of diagnostic context across multiple iterations, an order of magnitude larger than traditional optimization budgets.[15]

By performing counterfactual diagnosis across these raw execution traces, the Meta-Harness identifies the precise root cause of a failure—whether it is a flawed prompt instruction, an overly restrictive output schema, or a missing dependency in the DAG.[33] The proposer then physically rewrites the system's own harness code, generating a new Archon YAML file or updating the memory extraction scripts.[16] It evaluates this new candidate harness against a search set, and if the modifications yield higher performance, the changes are permanently merged. This self-healing loop allows Project "Real Jarvis" to continuously evolve its own underlying codebase without requiring manual human intervention.[15]

Swarm Intelligence and Self-Regulation Logic (Pheromind Stigmergy)

The execution of individual nodes within an Archon workflow is frequently too complex for a single zero-shot LLM inference. To solve intricate logical problems, systems often deploy multiple agents to collaborate. However, standard multi-agent frameworks—where agents "chat" with one another by appending their outputs to a shared conversational log—suffer from catastrophic inefficiencies.[1]

The Token Furnace Problem and Stigmergic Coordination

As conversational agents exchange messages, the context window grows quadratically. An agent processing a dozen subtasks must carry all previous outputs in its active memory. By the twelfth subtask, the context window may have grown by a factor of 28, transforming the system into a "token furnace" where coordination overhead vastly exceeds actual computational work.[1]

To circumvent this architectural bottleneck, Project "Real Jarvis" employs **Pheromind**, a swarm intelligence framework based on stigmergy. Stigmergy is the mechanism by which social

insects, such as ants and termites, coordinate complex, NP-hard logistical problems without centralized control or direct communication.[28] Instead of talking to each other, individuals communicate indirectly by modifying their shared environment—specifically, by leaving pheromone trails that guide subsequent behavior.[28]

In the Pheromind architecture, sub-agents deployed by Archon operate in parallel as asynchronous tasks. They never exchange direct LLM-to-LLM tokens. Instead, their shared environment is the Cognee knowledge graph.[1] As agents traverse graph nodes, execute tools, or evaluate hypotheses, they leave "digital pheromones" on the graph's edges.[47]

Ternary Mathematics and Signal Regulation

To ensure these digital pheromones provide clear, unambiguous guidance to the swarm, Pheromind regulates the signals using balanced ternary mathematics, employing the discrete states $-1, 0, 1$.[30]

- **State $+1$ (Reinforcement / Super Hot):** When a worker agent successfully completes a sub-task, verifies a hypothesis, or finds a mathematically sound path, it deposits a $+1$ signal on the corresponding Cognee graph edge. This serves as positive reinforcement, creating an attractive gradient that signals other parallel agents to converge on and expand this line of reasoning.[30]
- **State 0 (Neutral Traversal):** Represents unverified exploration or baseline operational status. It indicates that a path is being traversed but has not yet yielded actionable success or failure.[30]
- **State -1 (Decay / Chilly Cold):** If an agent encounters a compilation error, detects a logical hallucination, or hits a dead-end within the Archon workflow, it immediately deposits a -1 signal. This serves as powerful negative feedback, creating a repulsive pressure gradient that instantly warns other agents to abandon the flawed trajectory, preventing the swarm from wasting compute cycles on failed hypotheses.[30]

Mathematical Implementation of Aggregation and Evaporation

For stigmergic coordination to remain adaptive, the environment cannot become permanently biased by early exploration. Therefore, digital pheromones must be subject to an evaporation mechanism that decays signals over time, allowing the system to abandon outdated solutions when new environmental parameters arise.[31]

The concentration of pheromones on a specific Cognee graph edge (i, j) at discrete time t is denoted as $\phi(i, j, t)$. The baseline evaporation function contributes to the variation of pheromone concentration as follows:

$$\Delta e(i,j,t) = -\epsilon\phi(i,j,t)$$

Where $\epsilon \in (0,1]$ represents the predetermined pheromone decay rate or evaporation coefficient.[49]

As the swarm operates, N individual agents traverse the graph, depositing ternary signals $\Delta\tau_k(i,j) \in \{-1,0,1\}$. The global update equation for the pheromone concentration on edge (i,j) for the next time step $t+1$ integrates both the decay of historical signals and the aggregation of new ternary depositions:

$$\phi(i,j,t+1) = (1-\epsilon)\phi(i,j,t) + \sum_{k=1}^{N} \Delta\tau_k(i,j)$$

By continuously calculating these values across the graph, the swarm achieves decentralized self-regulation. The -1 signals rapidly suppress faulty logic paths, while the $+1$ signals amplify optimal solutions.[31] The Meta-Harness proposer continuously monitors these pheromone density gradients across the Cognee graph. A dense accumulation of -1 signals around a specific Archon task node serves as the primary diagnostic trigger, alerting the Meta-Harness that a fundamental workflow rewrite is required.[31]

Safety, Observability, and State Logging (Convex.dev)

A system exhibiting the capabilities of Project "Real Jarvis"—autonomous self-modification, parallel tokenless swarm interactions, infinite memory paging, and spontaneous environmental interruptions—presents profound challenges for monitoring, debugging, and AI safety. Traditional application performance monitoring (APM) tools are insufficient, as they are designed to track deterministic API responses rather than dynamic logical reasoning traces or chaotic swarm intelligence behaviors.[51]

The inner loop of this system operates as a screaming chaos of thousands of parallel micro-actions and continuous environmental mutations.[29] To guarantee absolute observability and ensure that human operators can audit the system without disrupting the streamlined UX, the architecture leverages **Convex.dev** as its foundational logging and state management backend.[17]

Immutable, Reactive Telemetry Schema

Convex operates as a reactive database that treats backend state as pure, strictly typed TypeScript.[34] 100% of the system's inner workings are logged here, elevating cognitive processes from ephemeral background tasks to a first-class, immutable telemetry surface.[52]

Crucially, Convex employs Optimistic Concurrency Control (OCC), ensuring that thousands of simultaneous, highly parallel pheromone updates from the Pheromind swarm do not result in database write conflicts or race conditions.[53]

The system defines rigorous, end-to-end type-safe schemas utilizing Convex's schema builder to enforce data integrity.[54] The following table defines the core data structures utilized for telemetry persistence:

Convex Table Schema	**Indexed Fields**	**Monitored Data Points**	**Purpose / Function**
execution_traces	workflow_id, step_id	input_context, output_result, status (success/failure/pending)	Logs the exact inputs and outputs of every node within the Archon YAML DAGs. Required by the Meta-Harness for filesystem-based counterfactual diagnosis.
stigmergic_signals	node_source, node_target	ternary_value (-1, 0, 1), agent_id, timestamp	Records the deposition of digital pheromones by the swarm across Cognee graph edges. Facilitates tracking of hot paths and dead-ends.
recursive_thoughts	session_id	thought_trace, memory_page_fault (boolean), timestamp	Captures the internal reasoning chains of agents prior to execution, as well as tracking every instance MemGPT pages data in or out of active context.
harness_mutations	version_id	diff_patch, evaluation_score, rollback_hash	Logs every instance where the Meta-Harness proposer rewrites an Archon YAML file or system prompt, including a cryptographic hash for immediate rollback capabilities.

Real-Time Observability and the "Status Dashboard"

While the MCU-inspired voice interface presents a calm, collected facade to the user, human oversight requires immediate, transparent access to the underlying state. Convex powers a real-time "Status Dashboard" to fulfill this requirement.[18]

Because Convex is engineered for ubiquitous reactivity, the Status Dashboard subscribes directly to the database queries.[18] Whenever a Pheromind agent updates a stigmergic ternary signal, or the Meta-Harness successfully mutates an Archon YAML file, Convex automatically pushes the state update to the dashboard's user interface via continuous WebSockets.[18] This completely eliminates the need for manual browser refreshing, complex polling intervals, or cache invalidation logic.[18]

Human operators can visually monitor the real-time emergence of attractive gradients (+1) and repulsive gradients (-1) propagating through the Cognee knowledge graph. Furthermore, Convex Log Streams seamlessly integrate via the function_execution topic with enterprise observability platforms such as Axiom or Datadog.[35] This external integration enables infinite retention of historical cognitive logs and facilitates the creation of threshold-based alerting systems—notifying administrators immediately if the swarm exhibits aberrant recursive loops or begins to approach critical API rate limits.[35]

Conclusion

Project "Real Jarvis" transcends the superficial capabilities of contemporary LLM wrappers by architecting a deeply integrated, biologically and computationally inspired cognitive ecosystem. By abandoning open-ended prompting in favor of the Archon Harness Builder's deterministic orchestration, the system prevents execution drift and enforces rigorous validation standards. Working in symbiosis, MemGPT and Cognee obliterate the constraints of fixed context windows, providing a limitless, dynamically evolving memory architecture capable of both granular episodic recall and the cultivation of rapid, somatic workflow reflexes.

The integration of the tokenless Pheromind swarm solves the critical inefficiencies of multi-agent communication, allowing for highly parallelized, decentralized problem-solving governed by the mathematical elegance of ternary stigmergy. Crucially, the Meta-Harness guarantees the system's longevity and adaptability, empowering it to autonomously diagnose failures and rewrite its own foundational code by analyzing massive execution traces across the filesystem.

Grounded entirely by the immutable, reactive safety layer of Convex.dev, the immense complexity of this architecture remains fully observable, auditable, and secure. Ultimately, Project "Real Jarvis" succeeds in masking profound technical complexity behind an effortless, proactive voice-driven interface, bringing the autonomous, highly capable, and characteristically witty persona of its cinematic inspiration from the realm of fiction into operational reality.

Works cited

1. TEMM1E v3.0.0 — Stigmergic Swarm Intelligence for AI Agent Runtimes : r/Rag - Reddit, accessed April 16, 2026, https://www.reddit.com/r/Rag/comments/1rwzfxa/temm1e_v300_stigmergic_swarm_intelligence_for_ai/
2. The Three Memory Systems Every Production AI Agent Needs - TianPan.co, accessed April 16, 2026, https://tianpan.co/blog/long-term-memory-types-ai-agents
3. J.A.R.V.I.S. - Wikipedia, accessed April 16, 2026, https://en.wikipedia.org/wiki/J.A.R.V.I.S.
4. J.A.R.V.I.S. | Marvel Cinematic Universe Wiki, accessed April 16, 2026, https://marvelcinematicuniverse.fandom.com/wiki/J.A.R.V.I.S.
5. Jarvis (MCU) J.A.R.V.I.S AI Voice Generator | Fish Audio, accessed April 16, 2026, https://fish.audio/m/612b878b113047d9a770c069c8b4fdfe/
6. Marvel Cinematic Universe AI Voices for Audiobooks & Podfics - LyricWinter, accessed April 16, 2026, https://lyricwinter.com/voices/marvel-cinematic-universe
7. What Is the Archon Harness Builder? The Open-Source Framework for Custom AI Coding Workflows | MindStudio, accessed April 16, 2026, https://www.mindstudio.ai/blog/what-is-archon-harness-builder-ai-coding
8. What Is the Archon Harness Builder? The Open-Source Framework for Custom AI Coding Workflows | MindStudio, accessed April 16, 2026, https://www.mindstudio.ai/blog/what-is-archon-harness-builder
9. MemGPT: Towards LLMs as Operating Systems - NSF Public Access Repository, accessed April 16, 2026, https://par.nsf.gov/servlets/purl/10524107
10. MemGPT: Towards LLMs as Operating Systems - alphaXiv, accessed April 16, 2026, https://www.alphaxiv.org/overview/2310.08560
11. How Cognee Builds AI Memory for Agents, accessed April 16, 2026, https://www.cognee.ai/blog/fundamentals/how-cognee-builds-ai-memory
12. Memify - Cognee Documentation, accessed April 16, 2026, https://docs.cognee.ai/core-concepts/main-operations/memify
13. Stigmergic interaction in robotic multi-agent systems using virtual pheromones - DiVA portal, accessed April 16, 2026, https://www.diva-portal.org/smash/get/diva2:1887312/FULLTEXT01.pdf
14. (PDF) Ternary Mathematics and 3D Placement of Logical Elements Justification, accessed April 16, 2026, https://www.researchgate.net/publication/354026158_Ternary_Mathematics_and_3D_Placement_of_Logical_Elements_Justification
15. Meta-Harness: End-to-End Optimization of Model Harnesses - arXiv, accessed April 16, 2026, https://arxiv.org/html/2603.28052v1
16. Report on META-HARNESS - Hugging Face, accessed April 16, 2026, https://huggingface.co/blog/Svngoku/meta-harness-end-to-end-optimization-of-model
17. AI Agents | Convex Developer Hub, accessed April 16, 2026, https://docs.convex.dev/agents

18. Realtime, all the time - Convex, accessed April 16, 2026, https://www.convex.dev/realtime
19. Beyond Short-term Memory: The 3 Types of Long-term Memory AI Agents Need - MachineLearningMastery.com, accessed April 16, 2026, https://machinelearningmastery.com/beyond-short-term-memory-the-3-types-of-long-term-memory-ai-agents-need/
20. Synapses and Memory Storage | Request PDF - ResearchGate, accessed April 16, 2026, https://www.researchgate.net/publication/223977656_Synapses_and_Memory_Storage
21. Archon: The Open-Source AI Coding Harness for Agentic Workflows | Stork.AI, accessed April 16, 2026, https://www.stork.ai/blog/this-tool-tames-chaotic-ai-agents
22. GitHub - coleam00/Archon: The first open-source harness builder for AI coding. Make AI coding deterministic and repeatable., accessed April 16, 2026, https://github.com/coleam00/archon
23. MemGPT: A Deep Dive - Focal, accessed April 16, 2026, https://www.getfocal.co/post/unlocking-the-potential-of-language-models-with-memgpt-a-deep-dive
24. MemGPT with Real-life Example: Bridging the Gap Between AI and OS | DigitalOcean, accessed April 16, 2026, https://www.digitalocean.com/community/tutorials/memgpt-llm-infinite-context-understanding
25. MemGPT: Towards LLMs as Operating Systems - AWS, accessed April 16, 2026, https://readwise-assets.s3.amazonaws.com/media/wisereads/articles/memgpt-towards-llms-as-operati/MEMGPT.pdf
26. Cognee + Memgraph: How To Build An Intelligent Knowledge Graph Using Hacker News Data, accessed April 16, 2026, https://memgraph.com/blog/cognee-memgraph-integration-demo
27. How Cognee Builds AI Memory Layers with LanceDB, accessed April 16, 2026, https://www.lancedb.com/blog/case-study-cognee
28. A Formal Framework for Indirect Coordination Grounded in Distributed Ledger State - arXiv, accessed April 16, 2026, https://arxiv.org/html/2604.03997v1
29. Digital Pheromones: What Ants Know About Agent Coordination That We Don't, accessed April 16, 2026, https://www.distributedthoughts.org/2026-02-13-digital-pheromones-what-ants-know-about-agent-coordination/
30. (PDF) Ternary Logic Gates: Advancing Computing with -1, 0, 1 Base - ResearchGate, accessed April 16, 2026, https://www.researchgate.net/publication/383243392_Ternary_Logic_Gates_Advancing_Computing_with_-1_0_1_Base
31. Emergent Coordination in Multi-Agent Systems via Pressure Fields and Temporal Decay, accessed April 16, 2026, https://arxiv.org/html/2601.08129v3
32. Meta-Harness: End-to-End Optimization of Model Harnesses - arXiv, accessed April 16, 2026, https://arxiv.org/pdf/2603.28052

33. Meta-Harness - Yoonho Lee, accessed April 16, 2026, https://yoonholee.com/meta-harness/
34. get-convex/convex-agent-plugins: An plugin for cursor to empower it to build the best apps ever. - GitHub, accessed April 16, 2026, https://github.com/get-convex/convex-agent-plugins
35. Log Streams: Common uses - Stack by Convex, accessed April 16, 2026, https://stack.convex.dev/log-streams-common-uses
36. Convex | The backend platform that keeps your app in sync, accessed April 16, 2026, https://www.convex.dev/
37. [Iron Man 3] I think Jarvis has my favorite joke in the whole MCU. : r/marvelstudios - Reddit, accessed April 16, 2026, https://www.reddit.com/r/marvelstudios/comments/bplmt4/iron_man_3_i_think_jarvis_has_my_favorite_joke_in/
38. [MCU, Iron Man] Do Stark Industries UI's have the ability to understand sarcasm? - Reddit, accessed April 16, 2026, https://www.reddit.com/r/AskScienceFiction/comments/qsso5v/mcu_iron_man_do_stark_industries_uis_have_the/
39. I Forgot about Jarvis and having one of the funniest lines in MCU. : r/Marvel - Reddit, accessed April 16, 2026, https://www.reddit.com/r/Marvel/comments/1mcfwvb/i_forgot_about_jarvis_and_having_one_of_the/
40. Iron Man/Transcript | Moviepedia - Fandom, accessed April 16, 2026, https://movies.fandom.com/wiki/Iron_Man/Transcript
41. For me, this is my favorite scene in all of the MCU. An android speaking to an AI philosophically about the merits of humanity. : r/marvelstudios - Reddit, accessed April 16, 2026, https://www.reddit.com/r/marvelstudios/comments/apdldx/for_me_this_is_my_favorite_scene_in_all_of_the/
42. MemGPT: Unlimited Memory without Token Constraints for Generative AI Platforms, like GPT-4, LaMDA, PaLM, LLAMA, CLAUDE, and others | by Lawrence Teixeira | Medium, accessed April 16, 2026, https://medium.com/@lawrenceteixeira/memgpt-unlimited-memory-without-token-constraints-for-generative-ai-platforms-like-gpt-4-lamda-0c755ece7d05
43. Neuroplasticity, Memory and Sense of Self : An Epistemological Approach 9781935790976, 9781934542415 - DOKUMEN.PUB, accessed April 16, 2026, https://dokumen.pub/neuroplasticity-memory-and-sense-of-self-an-epistemological-approach-9781935790976-9781934542415.html
44. Sense, memory, and decision-making in the somatosensory cortical network | Request PDF, accessed April 16, 2026, https://www.researchgate.net/publication/230781288_Sense_memory_and_decision-making_in_the_somatosensory_cortical_network
45. AI Memory Layers & Context Engineering with Weighted ... - Cognee, accessed April 16, 2026, https://www.cognee.ai/blog/cognee-news/product-announcement-memory-weights

46. Beyond Recall: Building Persistent Memory in AI Agents with Cognee, accessed April 16, 2026, https://www.cognee.ai/blog/tutorials/beyond-recall-building-persistent-memory-in-ai-agents-with-cognee
47. Stigmergy – Knowledge and References - Taylor & Francis, accessed April 16, 2026, https://taylorandfrancis.com/knowledge/Engineering_and_technology/Systems_%26_control_engineering/Stigmergy/
48. Special Sense AI: Why Ternary Logic is the Future of Decentralized Intelligence - Medium, accessed April 16, 2026, https://medium.com/@rosan11sah/special-sense-ai-why-ternary-logic-is-the-future-of-decentralized-intelligence-3454006d062c
49. Islands of cooperation emerge by stigmergic interactions in iterated spatial games - PMC, accessed April 16, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC11244808/
50. (PDF) Stigmergy: from mathematical modelling to control - ResearchGate, accessed April 16, 2026, https://www.researchgate.net/publication/383748326_Stigmergy_from_mathematical_modelling_to_control
51. Tracing for agentic applications: a developer's guide | Rhesis AI Blog, accessed April 16, 2026, https://rhesis.ai/post/tracing-agentic-applications-developers-guide
52. AgentTrace: A Structured Logging Framework for Agent System Observability - arXiv, accessed April 16, 2026, https://arxiv.org/html/2602.10133v1
53. convex-best-practices | Skills Marke... - LobeHub, accessed April 16, 2026, https://lobehub.com/skills/waynesutton-convexskills-convex-best-practices
54. Schemas | Convex Developer Hub, accessed April 16, 2026, https://docs.convex.dev/database/schemas
55. Log Streams | Convex Developer Hub, accessed April 16, 2026, https://docs.convex.dev/production/integrations/log-streams/